



PUSS IN BOOTS

2D PLATFORMER

Maks Patafta

YT Link projekta: <https://youtu.be/s0BorwZznWQ?si=CJ2avzrnKkdXwABT>



Puss in Boots

- 2D platformer napravljen pixel art grafikom
- Glavni lik „Puss in Boots (Mačak u čizmama)
- Cilj: poraziti neprijatelje i sakupiti sve dijamante
- Funkcije: skakanje, melee borba, sakupljanje dijamanta
- Postoje dva levela igre koja su puna različitih prepreka i neprijatelja
- Mačak se može boriti prsa o prsa sa neprijateljem (Red Dog)
- Mačak se mora paziti neprijateljevog oružja i bodljikavih šiljaka koji su postavljeni širom mape



Kreiranje projekta

01

Otvorimo Unity Hub

02

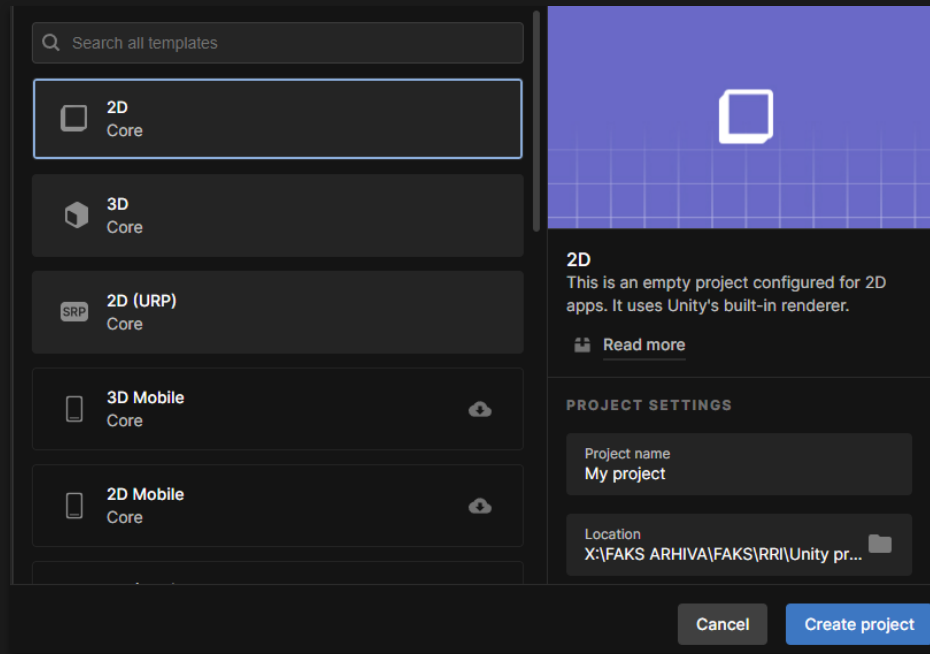
Odaberemo Template: 2D

03

Upišemo ime projekta

04

Odaberemo lokaciju na disku



Importiranje asseta



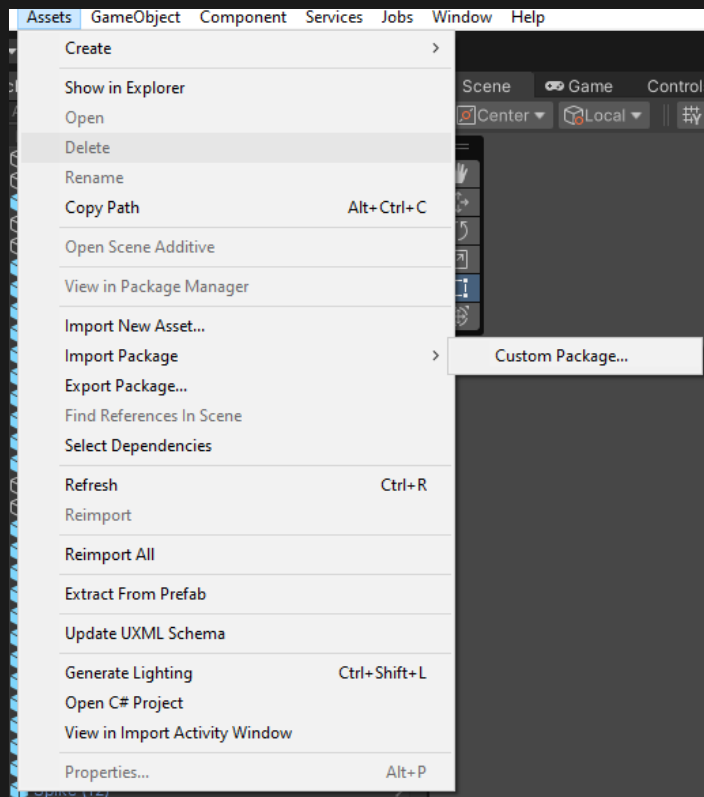
U importiranim assetima nalaze se Spriteovi i blokovi (materijali) pomoću kojih su napravljene scene

01

Assets > Import Package > Custom Package

02

Odaberemo u datotekama package u kojem se nalaze potrebni asseti



Podjela spriteova



Za sve sprite assete potrebno je definirati Pixels Per Unit na 16
Djeljive spriteove potrebno je „razrezati”

01

Inspector > Pixels Per Unit > 16

02

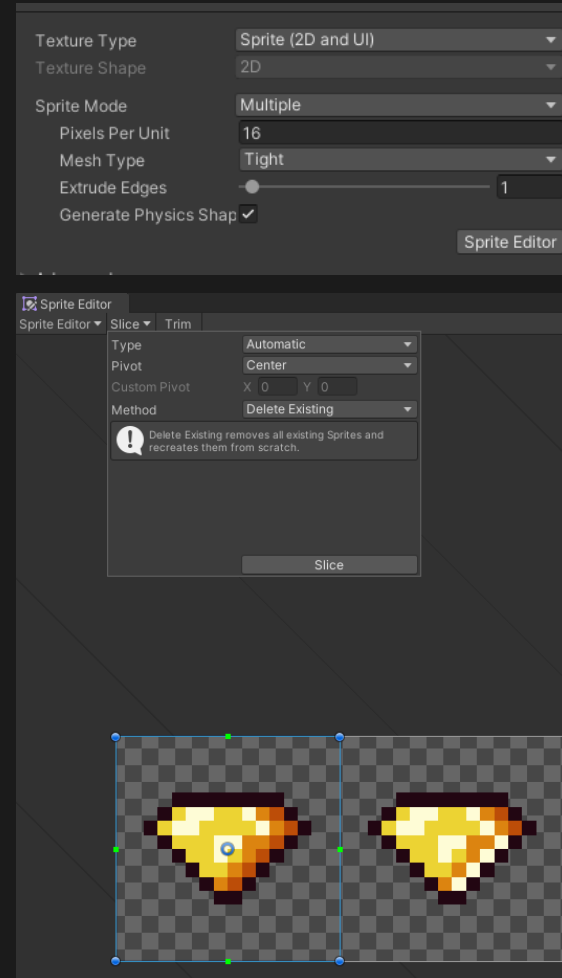
Inspector > Sprite Mode > Multiple

03

Inspector > Sprite Editor

04

Slice > Type > Automatic



Postavljanje Tilemapa

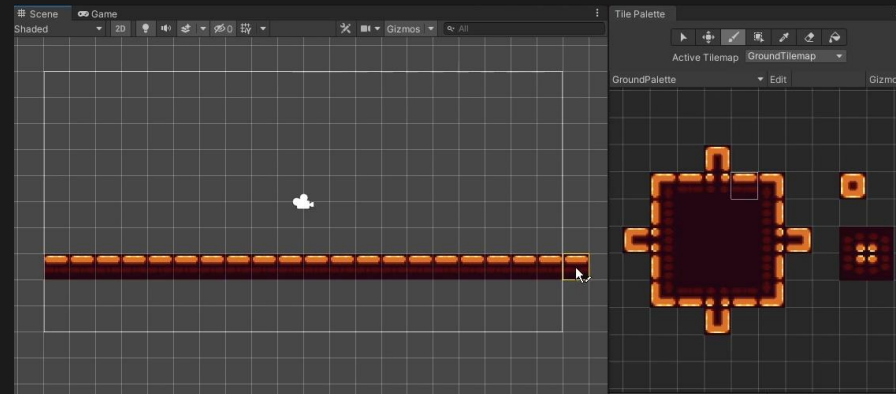
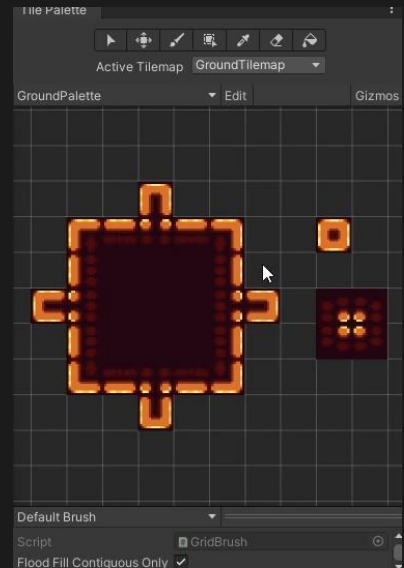
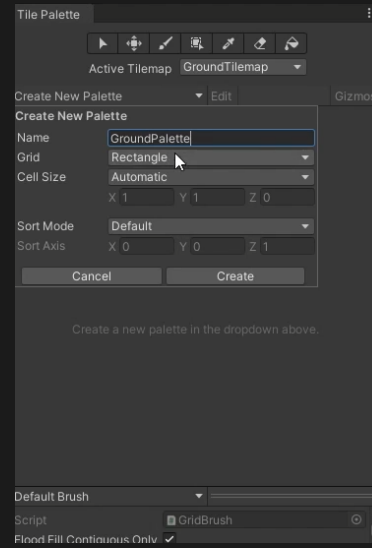
01 Hierarchy > 2D object > Tilemap > Rectangular

02 Postavljanje Tile Palette: Window > 2D > Tile Palette

03 Kreiranje nove palete: Create New Palette > GroundPalette > Create

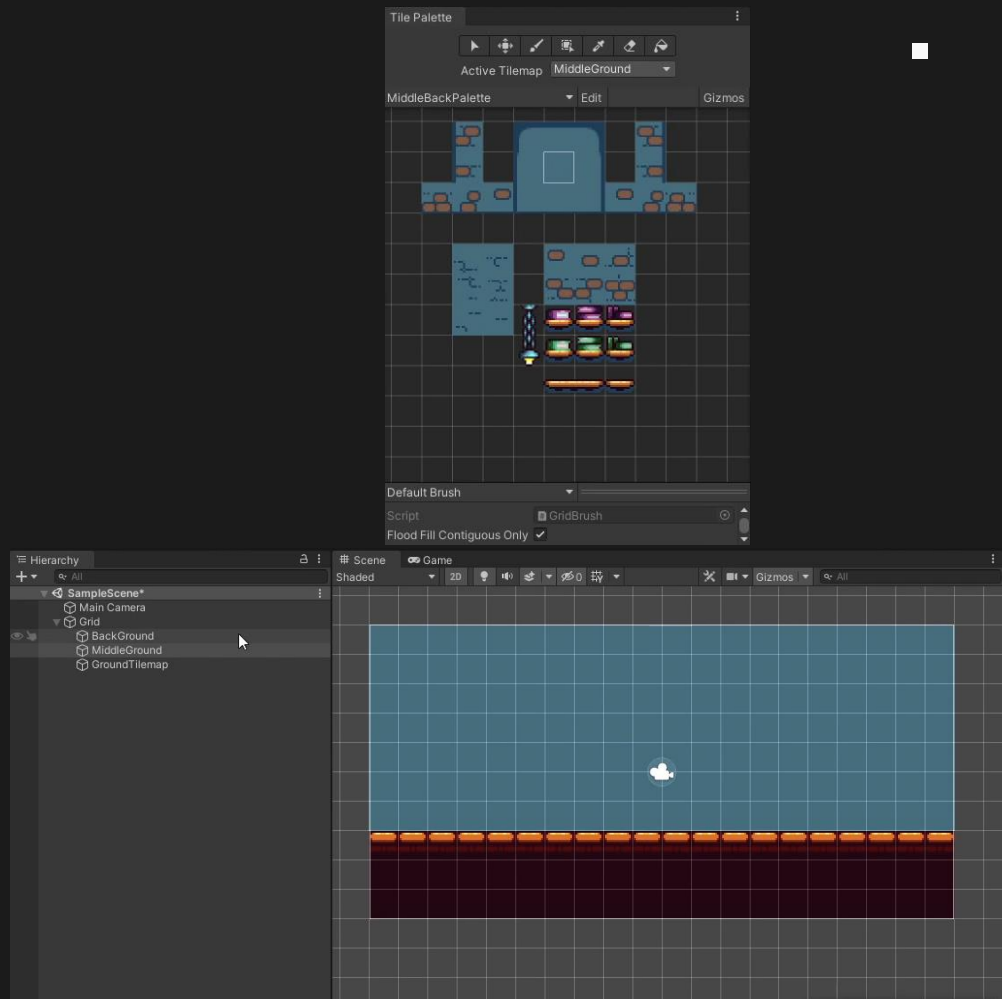
04 Dovući Sprite za platformu u Tile Paletu

05 Pomoću ikone kista mogu se postavljati kockice na scenu igre



Postavljanje Tilemapa

- 01** Kreiranje nove palete: Create New Palette > MiddlePalette > Create
- 02** Postaviti layere za pojedinu paletu: Inspector > Layer > ...
- 03** Postaviti kockice na scenu
- 04** Dodavanje svojstava za sudaranje da bi se igrač mogao kretati po površini: Inspector > Add Component > Rigidbody 2D > Body Type > Static
- 05** Inspector > Add Component > Composite Collider 2D > Geometry Type > Polygons



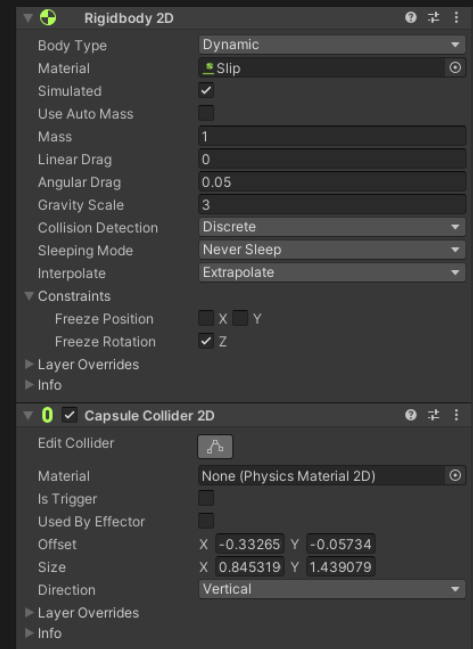
Kreiranje igrača

01

Dodati sprite igrača na scenu i promijeniti mu Sorting Layer na „Player”

02

Dodavanje fizikalnih osobnosti za sudaranje s objektima: Rigidbody 2D i Capsule Collider 2D



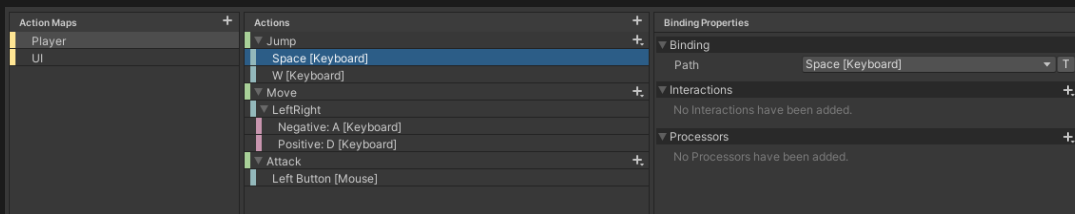
Kontrole kretanja igrača

01 Dodati Input System paket u Unity program: Window > Package Manager > Input System > Install

02 Kreiranje kontrola: Project > Input Actions

03 U Input Actions prozoru, napravimo dvije mape za objekte nad kojima radimo ulaze „Player” i „UI”

04 Dodajemo akcije pritiskom na gumb „+” i zatim u Binding Properties > Path, kliknemo na gumb na tipkovnici pomoću kojega želimo izvesti akciju



Kontrole kretanja igrača

01

Kreiranje skripte za Input Action unos:
Input Action > Inspector > Generate
C# Class > Apply

02

Igraču dodati „Player Input” skriptu i
dodijeliti mu Default Map „Player”

03

Kreirati novu skriptu „Gather Input” i
dodijelit ju igraču



Skripta „Gather Input” služi za
prikupljanje ulaznih akcija igrača

```
using UnityEngine;

public class GatherInput : MonoBehaviour
{
    private Controls myControls;

    public float valueX;
    public bool jumpInput;

    public bool tryAttack;

    [Unity Message | 0 references]
    private void Awake()
    {
        myControls = new Controls();
    }

    [Unity Message | 0 references]
    private void OnEnable()
    {
        myControls.Player.Move.performed += StartMove;
        myControls.Player.Move.canceled += StopMove;

        myControls.Player.Jump.performed += JumpStart;
        myControls.Player.Jump.canceled += JumpStop;

        myControls.Player.Attack.performed += TryToAttack;
        myControls.Player.Attack.canceled += StopTryToAttack;

        myControls.Player.Enable();
    }

    [Unity Message | 0 references]
    private void OnDisable()
    {
        myControls.Player.Move.performed -= StartMove;
        myControls.Player.Move.canceled -= StopMove;

        myControls.Player.Jump.performed -= JumpStart;
        myControls.Player.Jump.canceled -= JumpStop;

        myControls.Player.Attack.performed -= TryToAttack;
        myControls.Player.Attack.canceled -= StopTryToAttack;

        myControls.Player.Disable();
        //myControls.Disable();
    }
}
```

```
2 references
public void DisableControls()
{
    myControls.Player.Move.performed -= StartMove;
    myControls.Player.Move.canceled -= StopMove;

    myControls.Player.Jump.performed -= JumpStart;
    myControls.Player.Jump.canceled -= JumpStop;

    myControls.Player.Attack.performed -= TryToAttack;
    myControls.Player.Attack.canceled -= StopTryToAttack;

    myControls.Player.Disable();
    valueX = 0f;
}

3 references
private void StartMove(InputAction.CallbackContext ctx)
{
    valueX = ctx.ReadValue<float>();
}

3 references
private void StopMove(InputAction.CallbackContext ctx)
{
    valueX = 0;
}

3 references
private void JumpStart(InputAction.CallbackContext ctx)
{
    jumpInput = true;
}

3 references
private void JumpStop(InputAction.CallbackContext ctx)
{
    jumpInput = false;
}

3 references
private void TryToAttack(InputAction.CallbackContext ctx)
{
    tryAttack = true;
}
```

Kontrole kretanja igrača

01

Kreiranje skripte PlayerMoveControls i dodijeliti ju igraču

02

Postaviti zabranu rotacije igrača: Player > Inspector > Rigidbody2D > Constraints > Freeze Rotation > Z

03

Postaviti brzinu igrača: Player > Inspector > PlayerMoveControls > Speed > 3

04

Postaviti mogućnost skakanja igrača

```
public class PlayerMoveControls : MonoBehaviour
{
    public float speed;
    public float jumpForce;

    private GatherInput gI;
    private Rigidbody2D rb;
    private Animator anim;

    private int direction = 1;
    private bool doubleJump = true;
    public int additionalJumps = 2;
    private int resetJumpsNumber;

    public float rayLength;
    public LayerMask groundLayer;
    public Transform leftPoint;
    public Transform rightPoint;
    private bool grounded = true;

    public bool knockBack = false;
    public bool hasControl = true;

    void Start()
    {
        gI = GetComponent<GatherInput>();
        rb = GetComponent<Rigidbody2D>();
        anim = GetComponent<Animator>();

        resetJumpsNumber = additionalJumps;
    }
}
```

```
Unity Message | 0 references
void Update()
{
    SetAnimatorValues();
}

Unity Message | 0 references
private void FixedUpdate()
{
    CheckStatus();
    if (knockBack || hasControl==false)
        return;
    Move();
    JumpPlayer();
}

1 reference
private void Move()
{
    Flip();
    rb.velocity = new Vector2(speed * gI.valueX, rb.velocity.y);
}

1 reference
private void JumpPlayer()
{
    if(gI.jumpInput==true)
    {
        if(grounded)
        {
            rb.velocity = new Vector2(gI.valueX * speed, jumpForce);
            doubleJump = true;
        }
        else if(additionalJumps>0)
        {
            rb.velocity = new Vector2(gI.valueX * speed, jumpForce);
            doubleJump = false;
            additionalJumps--;
        }
    }
    gI.jumpInput = false;
}
```



Kreiranje animacije igrača

01

Window > Animation > Animation > Create > *Naziv animacije*

02

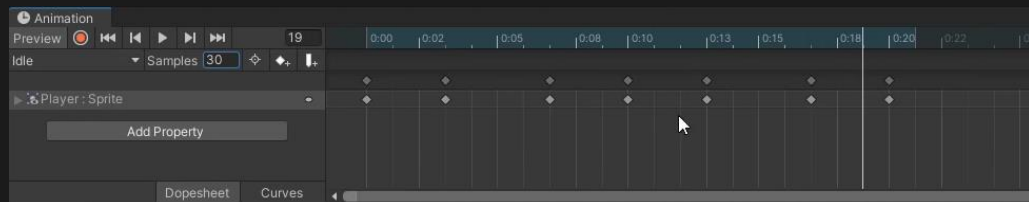
Dovući rascjepkane spriteove za animacije u Animation prozor

03

Podesiti brzinu Keyframe-ova

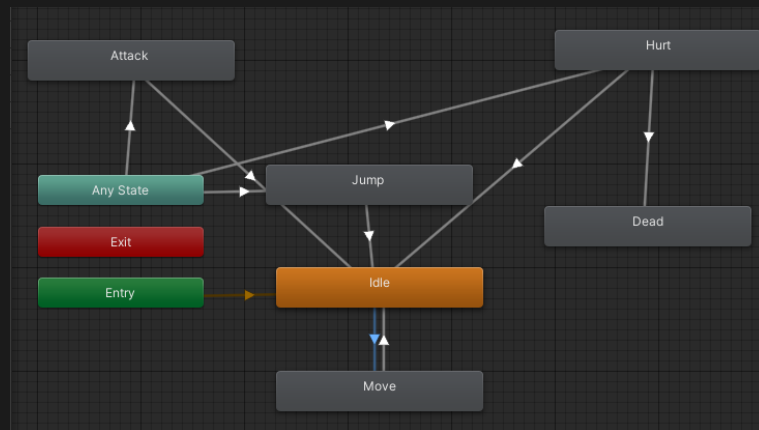
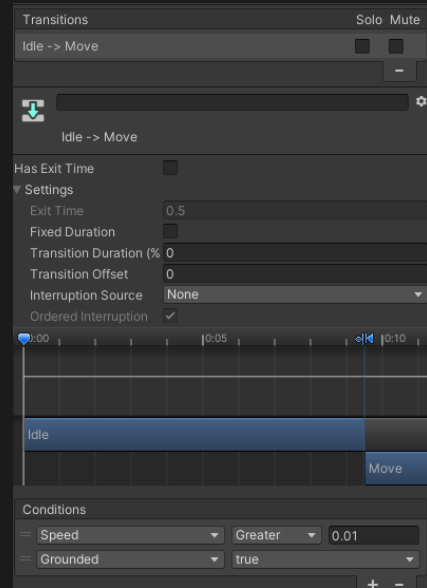
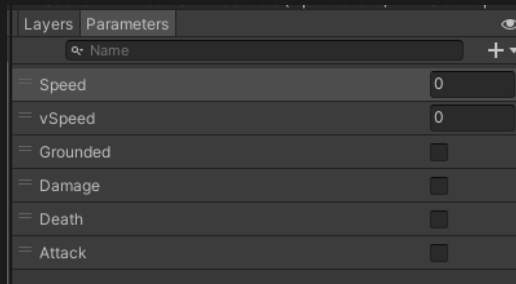


Za igrača postoje animacije: Idle, Move, Attack, Hurt, Jump



Tranzicija između animacija

- 01** Window > Animation > Animator
- 02** Postaviti „Idle” kao „Default state”
- 03** U Animator > Inspector > Settings, postaviti Transition Duration na 0, isključiti Has Exit Time i isključiti Fixed duration
- 04** Dodati parametre koji se pridodaju kao uvjeti kada se događa tranzicija između animacija: Animator > Parameters > + > ...
- 05** Postaviti vrijednosti dodatnih parametara u skripti „PlayerMoveControls”



Funkcija dvoskoka igrača

01

Window > Animation > Animator

02

U skripti „PlayerMoveControls” dodati mogućnosti jačine skoka (Jump Force) i broj skoka (Additional Jumps)

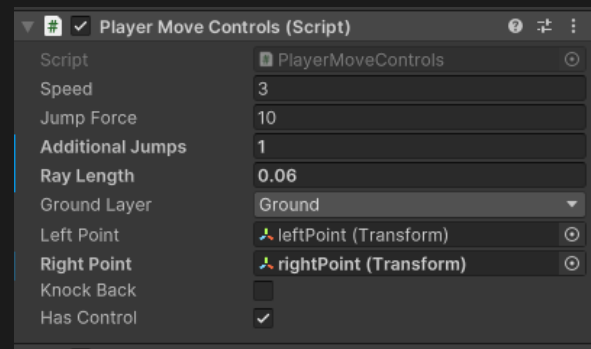
03

U funkciji se provjerava je li igrač na tlu, te ako je onda se može izvršiti skok.

04

Postavlja se brzina (velocity) igrača na novi vektor koji kombinira horizontalni pomak („gI.valueX*speed”) s jačinom skoka („jumpForce”)

```
private void JumpPlayer()
{
    if(gI.jumpInput==true)
    {
        if(grounded)
        {
            rb.velocity = new Vector2(gI.valueX * speed, jumpForce);
            doubleJump = true;
        }
        else if(additionalJumps>0)
        {
            rb.velocity = new Vector2(gI.valueX * speed, jumpForce);
            doubleJump = false;
            additionalJumps--;
        }
    }
    gI.jumpInput = false;
}
```



Praćenje kamere

01

Koristit ćemo Cinemachine opciju iz Unity-evog package managera. Window > Package Manager > „Cinemachine” > Install

02

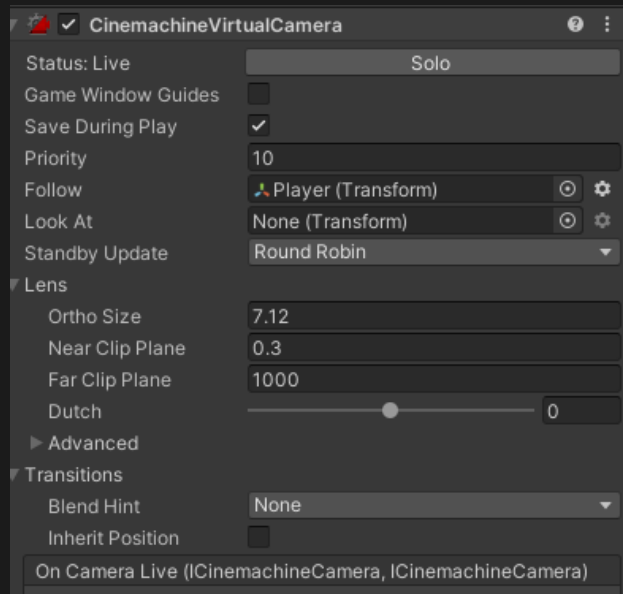
Kreirati Cinemachine 2D kameru. Nakon što se kreira kamera, na Main Camera dodaje se atribut „CinemachineBrain”. Hierarchy > Cinemachine > Create 2D Camera

03

Definirati praćenje kamere na objekt igrača. Cinemachine > Inspector > CinemachineVirtualCamera > Follow > Player

04

Postaviti veličinu kamere. Inspector > Lens > Orthographic Size > „7”



Praćenje kamere

01

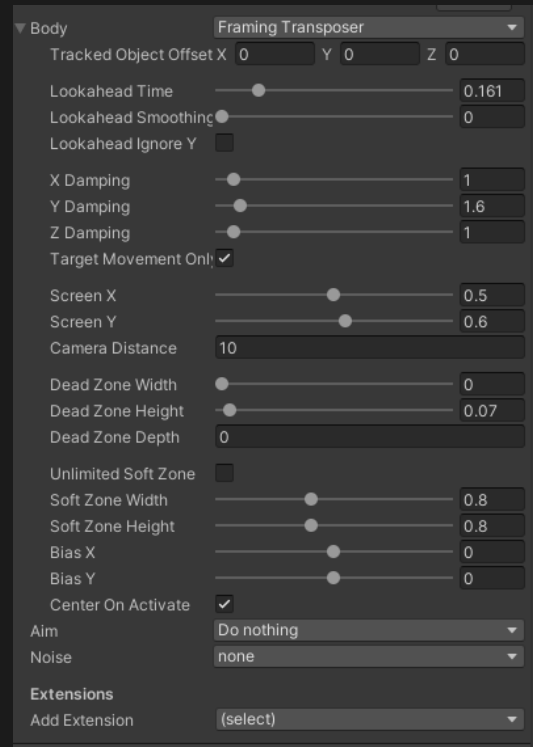
Na igraču potrebno je postaviti postavku za „glatko” praćenje kamere. Player > Inspector > Rigidbody2D > Interpolate > Extrapolate

02

Na virtualnoj kameri može se postaviti postavka koja nadgleda unaprijed s obzirom na igrača. VirtualCamera > Inspector > Cinemachine Virtual Camera > Body > Lookahead Time > 0.16

03

Postavljanje „mrtvih točaka” s obzirom na objekt, kada igrač skače kamera se ne pomiče sve dok ne dođe do „mrtve točke”. VirtualCamera > Inspector > Cinemachine Virtual Camera > Body > Dead Zone Height > 0.07



Pixel Perfect Camera



Za kreiranje 2D igrice pomoću pixel art grafike, često se koristi opcija za kamere Pixel Perfect Camera koja podjednako učitava piksele i daje sceni bolji izgled.

01

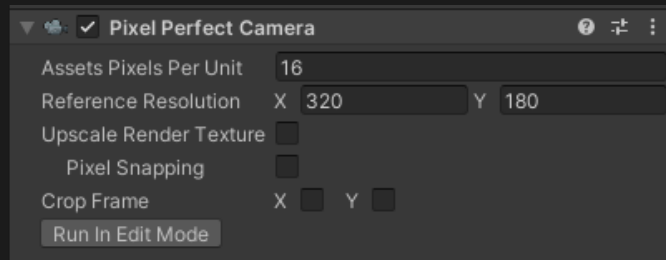
Main Camera > Inspector > Add Component > Pixel Perfect Camera

02

Postaviti koliko se piksela prikazuje kao jedan „blok”. U ovoj igri svi blokovi su rascjepkani na veličinu 16. Main Camera > Inspector > Pixel Perfect Camera > Assets Pixels Per Unit > 16

03

Virtual Camera > Inspector > Add Component > Pixel Perfect Camera



Postavljanje granica kamere



Kada kamera prati igrača, stavlja ga na sredinu scene. Ne želimo da se na početku ili kod krajeva scene prikazuje ostatak scene koji je samo podloga i zato definiramo granice do kojih kamera prati igrača.

01

Virtual Camera > Inspector > Add Component > Cinemachine Confiner

02

Cinemachine Confiner traži oblik koji uzima kao granice scene pa zato se kreira novi objekt samo za granice. Hierarchy > Create Empty > „CameraBoundaries”

03

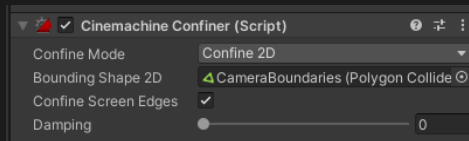
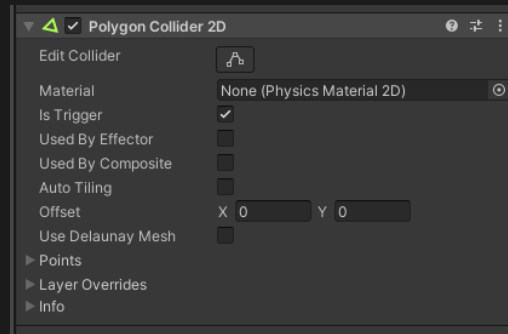
CameraBoundaries > Inspector > Add Component > Polygon Collider 2D > Edit Collider

04

CameraBoundaries > Inspector > Polygon Collider 2D > IsTrigger

05

VirtualCamera > Inspector > Cinemachine Confiner > Bounding Shape 2D > CameraBoundaries



Sakupljanje dijamanta

01

Potrebno je dodati sprite dijamanta na scenu i dodati mu fizikalna svojstva. Gem > Inspector > Add Component > Circle Collider 2D > IsTrigger

02

Gem > Inspector > Add Component > Rigidbody 2D > Body Type > Kinematic

03

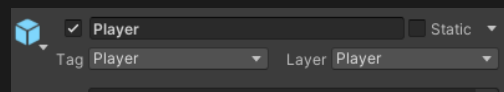
Kreirati skriptu za sakupljanje dijamanta. Gem > Inspector > Add Component > „Gem” > New Script

04

Postaviti oznaku „Player” na igrača kako bi lakše raspoznali objekte u skriptama. Player > Inspector > Tag > „Player”

05

U skripti napiši logiku da kada igrač dođe u koliziju s dijamantom, dijamant nestaje sa scene ali se nalazi u hijerarhiji

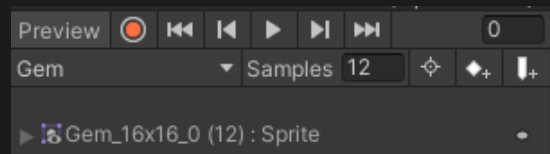


```
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Player"))
    {
        GetComponent<SpriteRenderer>().enabled = false;
        GetComponent<CircleCollider2D>().enabled = false;
        //Destroy(gameObject);
    }
}
```

Animacija dijamanata

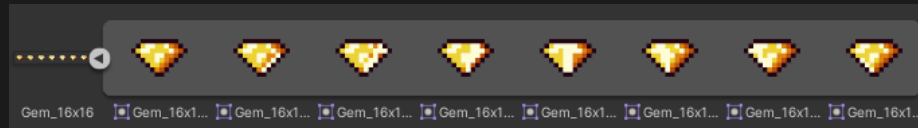
01

Dovući sprite-ove za dijamante u
Window > Animation > Animation



02

Kreirati novu animaciju. Animation > Create
new clip > „Gem”

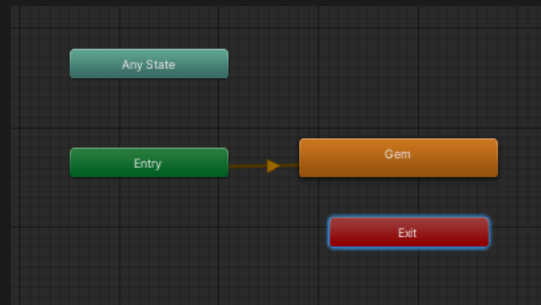


03

Postaviti brzinu izmjene keyframe-ova.
Samples > 12

04

U Window > Animation > Animator povezati
Entry sa Gem



Kreiranje UI brojača dijamanta

01

Hierarchy > + > UI > Image > „Gem”
Kreiranjem UI elemenata kreirao se roditeljski objekt „Canvas” i „Event System”

02

Canvas > Inspector > Canvas Scaler >
Reference Resolution > 960 x 540

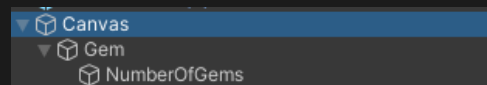
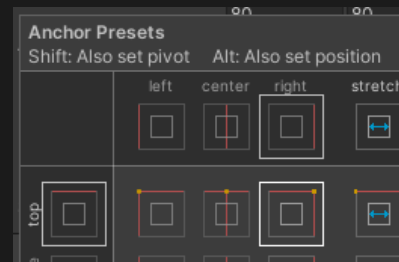
03

Postaviti sliku dijamanta na UI slici. Canvas >
Gem > Inspector > Source Image > „Gem”

04

Postaviti poziciju ikone dijamanta s obzirom na
izlazni ekran. Canvas > Gem > Inspector >
Rect Transform > Anchor Presets > Top Right

Dodati tekst kao dijete koji će prikazivati broj
dijamanata. Gem > UI > Text



Prikazivanje broja dijamanta

01

Kreirati novu skriptu za sakupljanje objekata.
Player > Add Component > „PlayerCollectibles”
> New Script

02

U skripti definirati da kada se objekt pokupi,
brojač se poveća za 1

03

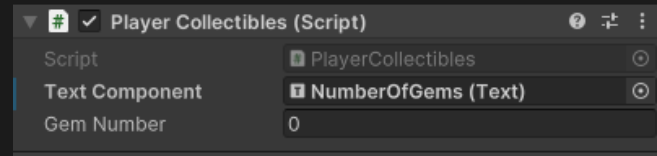
Postaviti varijable Text Component i Gem
number na javno

04

Player > Inspector > Player Collectibles > Text
Component > „NumberOfGems”

05

Player > Inspector > Player Collectibles >
Gem number > 0



```
public class PlayerCollectibles : MonoBehaviour
{
    public Text textComponent;
    public int gemNumber;

    [Unity Message | 0 references]
    void Start()
    {
        UpdateText();
    }

    [2 references]
    private void UpdateText()
    {
        textComponent.text = gemNumber.ToString();
    }

    [1 reference]
    public void GemCollected()
    {
        gemNumber += 1;
        UpdateText();
    }
}
```

Health Damage

01

Kreiraj objekt koji je dijete objektu Player i dodaj mu fizikalna svojstva. Player > Create Empty > „Player Stats” > Inspector > Polygon Collider 2D

02

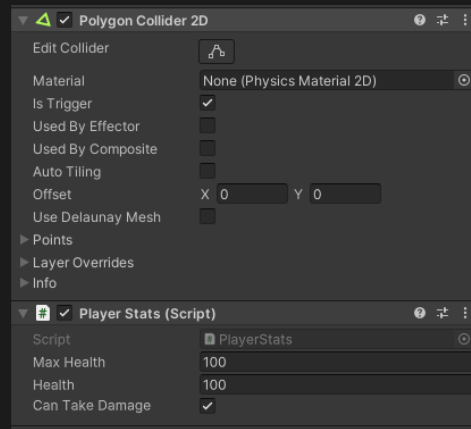
Kreiranje novog objekt je zbog toga da se dva collidera ne izmješaju. Player Stats > Inspector > Polygon Collider 2D > Is Trigger

03

Player Stats > Inspector > Layer > Add Layer > „PlayerStats”

04

Kreiranje skripte za logiku primanja štete. Player Stats > Inspector > Add Component > „PlayerStats” > New Script



```
public void TakeDamage(float damage)
{
    if(canTakeDamage)
    {
        health -= damage;
        anim.SetBool("Damage", true);
        playerMove.hasControl = false;

        UpdateHealthUI();
        pAC.ResetAttack();

        if (health <= 0)
        {
            GetComponent<PolygonCollider2D>().enabled = false;
            GetComponentInParent<GatherInput>().DisableControls();

            PlayerPrefs.SetFloat("HealthKey", maxHealth);
            GameManager.ManagerRestartLevel();
        }

        StartCoroutine(DamagePrevention());
    }
}
```

Kreiranje bodlji



Kreiranje bodlji koje mogu nanositi štetu ako igrač dođe u koliziju s njima. Bodlje su statički element koji se ne kreće po sceni, ali mora imati fizikalna svojstva.

01

Uvezi sprite bodlji („Spike“)

02

Spike > Inspector > Add Component > Rigidbody2D > Body Type > Kinematic

03

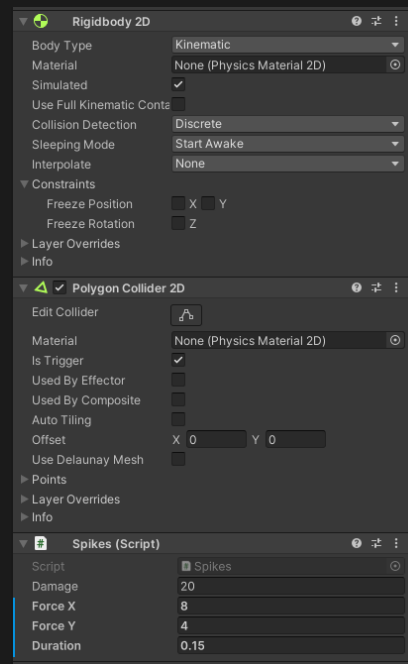
Spike > Inspector > Add Component > PolygonCollider 2D > Is Trigger

04

Spike > Inspector > Layer > Add Layer > „Enemy Attack“

05

Kreirati novu skriptu i postaviti broj koliko štete radi bodlja prilikom kolizije i koliko je vrijeme knockوتا. Spike > Add Component > „Spikes“ > New Script



```
{
    public float damage;

    public float forceX;
    public float forceY;
    public float duration;

    // Unity Message | 0 references
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Stats"))
        {
            collision.GetComponent<PlayerStats>().TakeDamage(damage);
            PlayerMoveControls playerMove = collision.GetComponentInParent<PlayerMoveControls>();

            StartCoroutine(playerMove.KnockBack(forceX, forceY, duration, transform));
        }
    }
}
```


Kreiranje animacije smrti

01

Kreiranje animacije kada igrač počinje dobivati štetu.
Animation > New Animation > „Hurt”

02

Kreiranje animacije umiranja. Animation > New Animation > „Death”

03

Animation > Samples > 15

04

U Animatoru postaviti veze tako da igrač može doći do oblika „Hurt” iz bilo kojeg drugog stanja, a do stanja „Death” dolazi samo od oblika „Hurt”

05

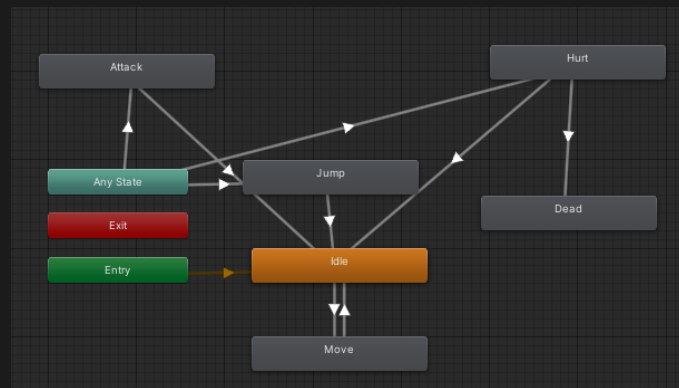
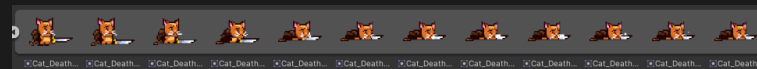
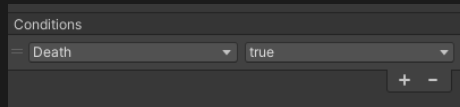
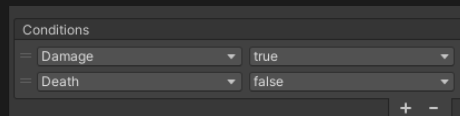
Kreiranje dva bool parametra „Damage” i „Death”

06

Dodijeliti parametre na vezu Any State > Hurt > Damage – True i Death – False

07

Za „Death” animaciju, parametar Death mora biti True



Kreiranje UI Health Bara

01

HealthBar će cijelo vrijeme prikazivati koliko je Healtha ostalo igraču i ažurirat će se ukoliko igrač prima štetu.

02

Kao objekt dijete stvorimo sliku. Create > UI > Image > „HealthBar”

03

HealthBar > Inspector > SourceImage > HealthBar prazni > Anchor Presets > Top Left

04

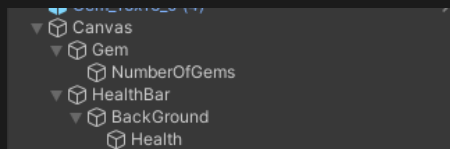
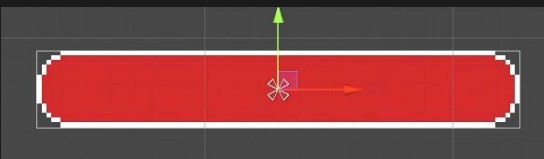
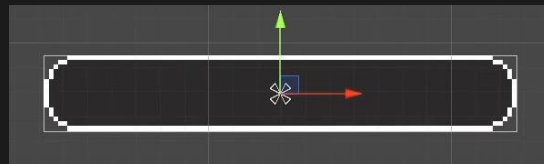
Dodajmo još jednu sliku kao objekt dijete HealthBar-u koji će predstavljati pozadinu. Create > UI > Image > „Background”

05

Dodajmo još jednu sliku kao objekt dijete Background-u koji će predstavljati količinu preostalih health-a. Create > UI > Image > „Health”

06

Health > Inspector > Color > Red



Kreiranje UI Health Bara

01

Postavljanje da se oduzimanje Healtha događa s desna prema lijevo. Health > Inspector > Image > Fill Origin > Left

02

Dodavanje ikone srca na kraj HealthBara. Ono je objekt dijete Healt Bar-u.

03

U skripti „PlayerStats” potrebno je napisati funkciju koja će oduzimati štetu na HealthBaru. U skripti se na atributu fillAmount puni/prazni crvena boja odnosno „Health”. Ta funkcija se poziva kada igrač primi štetu.

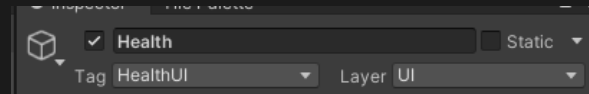
04

Objektu „Health” potrebno je dodati oznaku „HealthUI” i Layer „UI” kako bi ga mogli lakše referencirati u skripti



```
3 references
public void UpdateHealthUI()
{
    healthUI.fillAmount = health / maxHealth;
}
```

```
healthUI = GameObject.FindGameObjectWithTag("HealthUI").GetComponent<Image>();
UpdateHealthUI();
```



Pokupljanje srca



Kada igrač primi štetu, on gubi Health-e. Pokupljanjem objekta srca, igrač se može liječiti (heal) za određeni broj health-i.

02

Na scenu dovučemo sprite srca i preimenujemo ga u „Heart”.

03

Dodajemo fizikalna svojstva. Heart > Inspector > Add Component > Rigidbody 2D > Body Type > Kinematic

04

Heart > Inspector > Add Component > Circle collider 2D > Is Trigger

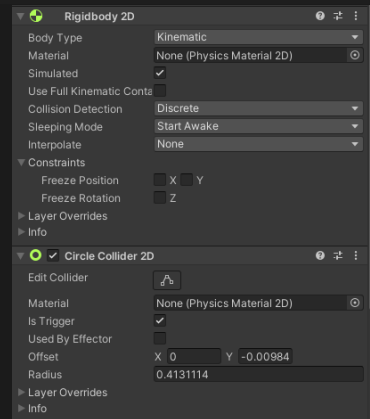
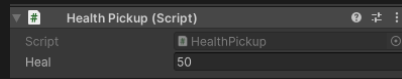
05

Dodajemo skriptu za pokupljanje srca u kojoj je napravljena logika da kada igrač dolazi u koliziju s objektom, da objekt nestaje i povećava „Health” igraču koji se odmah vidi na Health Bar-u. U skripti PlayerStats napišemo funkciju Heal. Heart > Inspector > Add Component > „Health Pickup” > New script

```
Unity Message | 0 references
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("Player"))
    {
        PlayerStats playerStats = collision.GetComponentInChildren<PlayerStats>();

        /* if (playerStats.health == playerStats.maxHealth)
            return; */

        playerStats.IncreaseHealth(heal);
        Destroy(gameObject);
    }
}
```



```
1 reference
public void IncreaseHealth(float heal)
{
    health += heal;

    if(health > maxHealth)
        health = maxHealth;

    UpdateHealthUI();
}
```

Kreiranje neprijatelja



Neprijatelj je crveni pas koji ima štit od bodlja kojima može nanijeti štetu našem igraču. Neprijatelj hoda lijevo-desno i ne napada direktno igrača, već samo patrolira. Ako se igrač zabije u neprijateljeve bodlje, onda mu se nanosi šteta.

01

Dovući sprite neprijatelja na scenu i preimenovat ga u `PatrollingEnemy`.

02

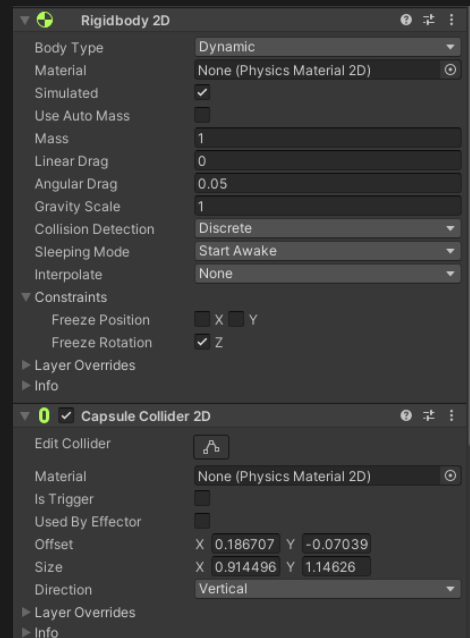
Dodati mu fizikalna svojstva `Rigidbody2D` i `Capsule Collider2D` koji nije trigger.

03

Dodati `Sorting Layer Enemy`

04

Kreirati animaciju hodanja neprijatelja.



Kretanje neprijatelja

01

Kreirati skriptu Enemy i skriptu PatrollingEnemy. Skripta PatrollingEnemy će biti skripta dijete skripte Enemy te će nasljeđivati sve funkcionalnosti roditeljske skripte.

02

Kreirati ćemo dva kruga koji će predstavljati granice do kojih neprijatelj može hodati. Gornji krug predstavlja granicu ako se ispred njega nalazi zid, a donji krug ako se ispred njega ne predstavlja dno. Kada gornji krug dodirne zid, ili donji krug ne osjeti dno, tada se neprijatelj okreće u drugom smjeru

03

U skripti PatrollingEnemy definirati kada se neprijatelj okreće i inicijalizirati varijable za detekciju zidova



```
public class PatrollingEnemy : Enemy
{
    public float speed;
    private int direction = -1;

    public Transform groundCheck;
    public Transform wallCheck;
    public LayerMask layerToCheck;

    private bool detectGround;
    private bool detectWall;

    public float radius;
```

```
1 reference
private void Flip()
{
    detectGround = Physics2D.OverlapCircle(groundCheck.position, radius, layerToCheck);
    detectWall = Physics2D.OverlapCircle(wallCheck.position, radius, layerToCheck);

    if(detectWall || detectGround == false)
    {
        direction *= -1;
        transform.localScale = new Vector3(-transform.localScale.x, 1, 1);
    }
}
```

```
Unity Message | 0 references
private void OnDrawGizmos()
{
    Gizmos.DrawSphere(groundCheck.position, radius);
    Gizmos.DrawSphere(wallCheck.position, radius);
}
```

```
Unity Message | 0 references
public class Enemy : MonoBehaviour
{
    public float health;

    protected Rigidbody2D rb;
    protected Animator anim;

    Unity Message | 0 references
    private void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        anim = GetComponent<Animator>();
    }

    1 reference
    public void TakeDamage(float damage)
    {
        health -= damage;
        HurtSequence();
        if(health <= 0)
        {
            DeathSequence();
        }
    }

    2 references
    public virtual void HurtSequence()
    {
    }

    2 references
    public virtual void DeathSequence()
    {
    }
}
```



Kretanje neprijatelja

01

Kreirati dva objekta prazna objekta djeteta na PatrollingEnemy koji će poslužiti kao indikatori „groundCheck” i „wallCheck”

02

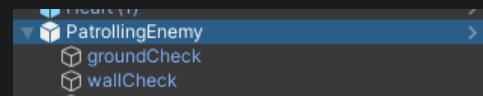
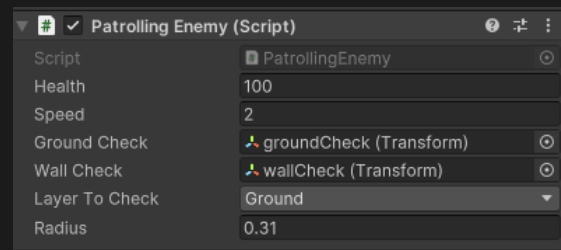
Dodijeliti indikatore u varijable na PatrollingEnemy skripti

03

PatrollingEnemy > Inspector > PatrollingEnemy Script > LayerToCheck > Ground

04

Postaviti varijable u skripti „Health”, „Speed” i „Radius”



Napad neprijatelja

01

Kreirati objekt dijete na PatrollingEnemy s nazivom „Attack” i dodijeliti mu sudarač. PatrollingEnemy > Create Empty > „Attack” > Inspector > Polygon Collider 2D > IsTrigger

02

Attack > Layer > EnemyAttack

03

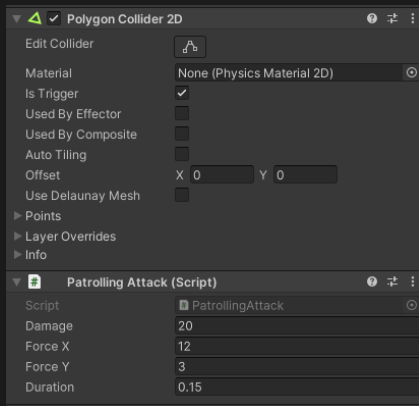
Podesiti granice Collidera tako da obuhvaća samo oružje

04

Kreirati skriptu PatrollingAttack koja nasljeđuje svojstva roditeljske skripte EnemyAttack.

05

U skripti PatrollingAttack potrebno je definirati koliko štete se radi našem igraču i koliko je trajanje knockbacka igrača kada se sudari s neprijateljem



```
public class EnemyAttack : MonoBehaviour
{
    public float damage;
    protected PlayerStats playerStats;

    // Unity Message | 0 references
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Stats"))
        {
            playerStats = collision.GetComponent<PlayerStats>();
            playerStats.TakeDamage(damage);

            SpecialAttack();
        }
    }

    // 2 references
    public virtual void SpecialAttack()
    {
    }
}
```

```
// Unity Script (1 asset reference) | 0 references
public class PatrollingAttack : EnemyAttack
{
    PlayerMoveControls playerMove;
    public float forceX;
    public float forceY;
    public float duration;

    // 2 references
    public override void SpecialAttack()
    {
        playerMove = playerStats.GetComponentInParent<PlayerMoveControls>();
        StartCoroutine(playerMove.KnockBack(forceX, forceY, duration, transform.parent));
    }
}
```


Napad neprijatelja

Skripta „EnemyAttack” predstavlja funkcionalnost napada neprijatelja u igri. Metoda OnTriggerEnter2D poziva se kada drugi objekt uđe u collider (kružni opseg) ovog objekta. Ako je objekt označen tagom "Stats", dohvaća se komponenta PlayerStats iz objekta s kojim se sudario neprijatelj. Zatim se poziva metoda

TakeDamage(damage) na komponenti PlayerStats, koja bi trebala smanjiti zdravlje igrača za iznos označen u varijabli damage. Nakon toga se poziva metoda SpecialAttack(), koja je definirana kao virtualna i može se zamijeniti u podklasama ove klase.

Skripta „PatrollingEnemy” proširuje funkcionalnosti napada neprijatelja. Varijabla „forceX” i „forceY” su Jačina udarca po X-osi, odnosno Y-osi koja će se primijeniti na igrača prilikom napada. Metoda SpecialAttack dohvaća komponentu PlayerMoveControls iz playerStats, koja bi trebala biti roditeljski objekt igrača. Zatim se pokreće korutina „StartCoroutine” koja služi za odbacivanje" igrača unatrag kada ga neprijatelj udari. Parametri koji se šalju metodi KnockBack() su jačina udarca po X i Y osi, trajanje odboja te roditeljski objekt neprijatelja koji izvodi udarac.

```
public class EnemyAttack : MonoBehaviour
{
    public float damage;
    protected PlayerStats playerStats;

    [Unity Message | 0 references]
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("Stats"))
        {
            playerStats = collision.GetComponent<PlayerStats>();
            playerStats.TakeDamage(damage);

            SpecialAttack();
        }
    }

    [2 references]
    public virtual void SpecialAttack()
    {
    }
}
```

```
[Unity Script | 1 asset reference] | 0 references
public class PatrollingAttack : EnemyAttack
{
    PlayerMoveControls playerMove;
    public float forceX;
    public float forceY;
    public float duration;

    [2 references]
    public override void SpecialAttack()
    {
        playerMove = playerStats.GetComponentInParent<PlayerMoveControls>();
        StartCoroutine(playerMove.KnockBack(forceX, forceY, duration, transform.parent));
    }
}
```

Promjena levela



Promjena levela moguća je jedino kada igrač skupi sve dijamante koji se nalaze na sceni. Promjena levela se izvodi kada igrač „prođe” kroz vrata.,



Kreiraj vrata pomoću kojih se promjeni level i dodaj fizikalna svojstva Rigidbody 2D i Box Collider 2D koji je trigger.



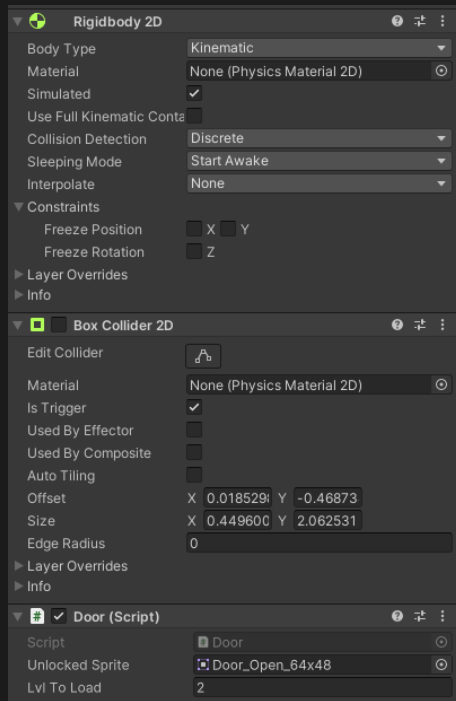
Kreiraj pripadajuću skriptu „Door” koja obavlja funkcionalnosti promjene levela.



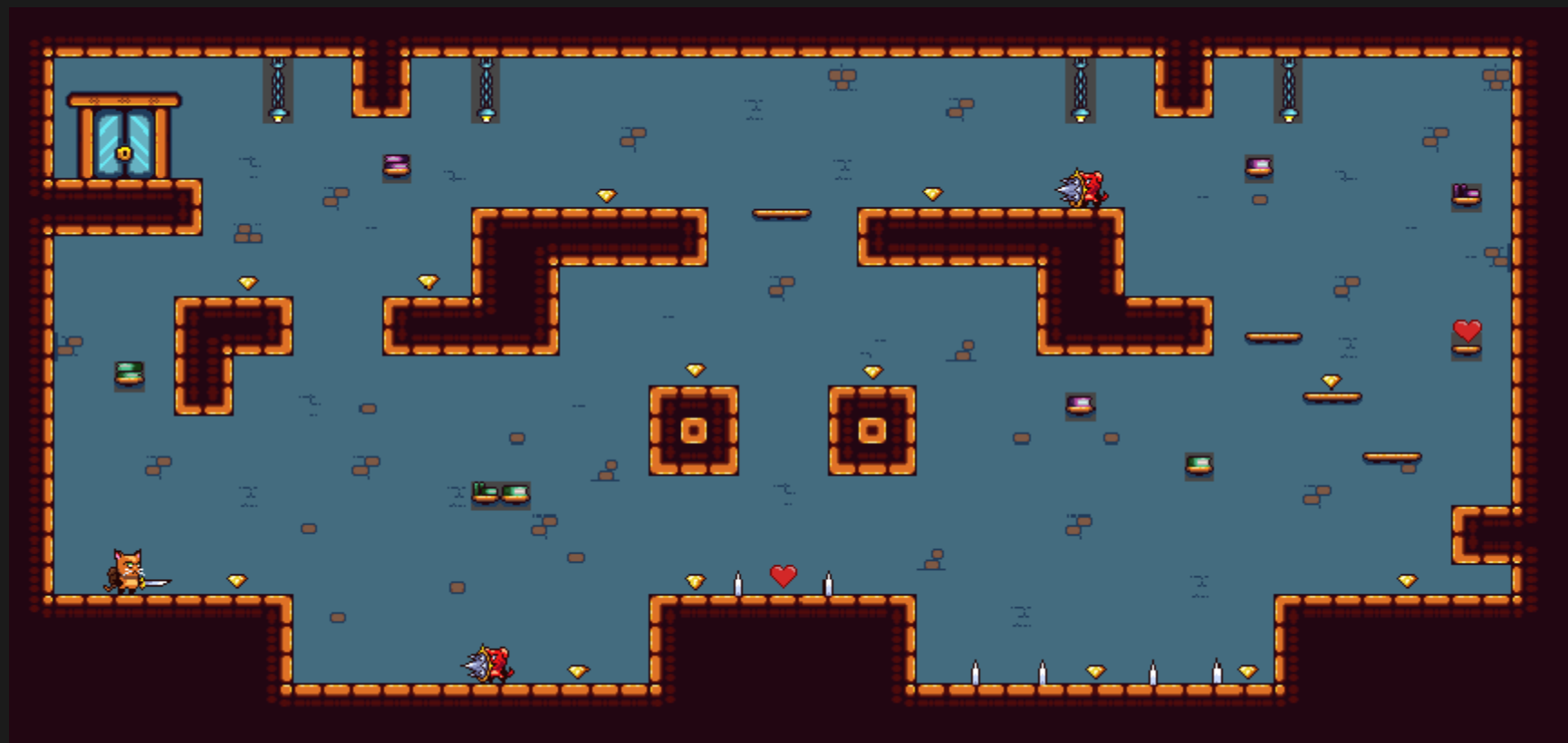
Kreiraj dvije nove scene: „Level2” i „Main Menu”. Project > Scenes > Level1 > Duplicate



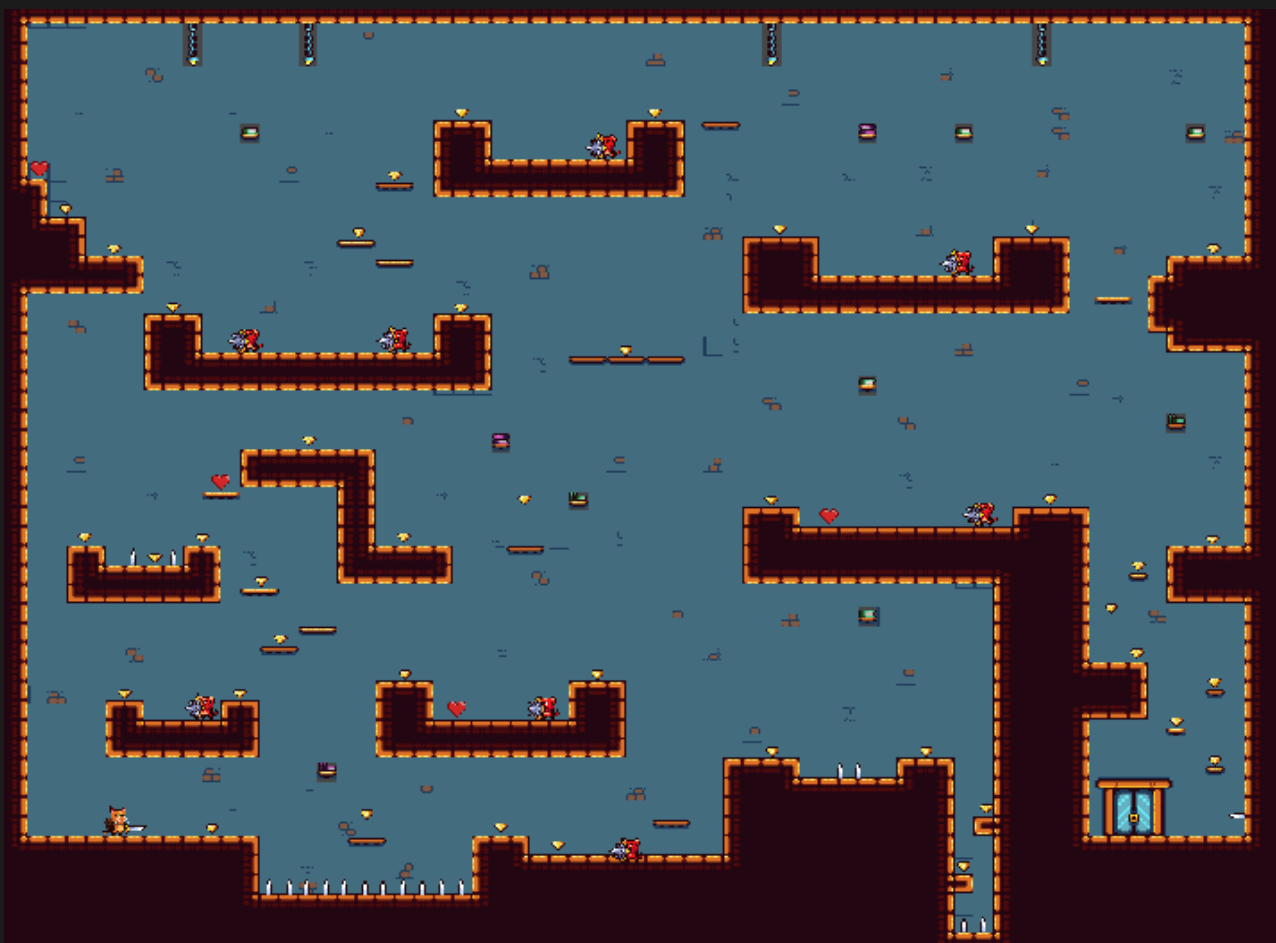
Dodajte sve napravljene scene, gdje svaka scena ima svoj poredak. Prvo treba biti scena Main Menu i zatim Level1 pa Level2
File > Build Settings > Add Open Scenes



Dizajn levela 1



Dizajn levela 2



Promjena levela

Skripta nazvana „Door” predstavlja funkcionalnost promjene razine levela u igri.

Varijable: ‘unlockedSprite’: Sprite koji će se prikazati na vratima nakon što se otključaju. ‘lvlToLoad’: Broj razine koja će se učitati nakon što igrač prođe kroz vrata. ‘boxCol’: Referenca na komponentu BoxCollider2D

U Start() metodi dohvaća se komponenta BoxCollider2D i registriraju se vrata u ‘Game Manageru’ kako bi se pratilo stanje vrata u igri.

Metoda OnTriggerEnter2D se poziva kada drugi objekt uđe u collider vrata. Ako je igrač prošao kroz vrata, onemogućuje se boxCol (kolizija vrata) kako bi se spriječilo ponovno prolazak kroz njih. Nakon što igrač dođe do vrata, onemogućuju mu se kontrole. Health igrača se pohranjuje u ‘PlayerPrefs’ pod ključem ‘HealthKey’ kako bi se sačuvali Health-i igrača s prošlog levela. Na kraju se poziva GameManager.ManagerLoadLevel kako bi se učitala nova razina igre.

Metoda UnlockDoor() otključava vrata tako što postavlja njihov sprite na unlockedSprite i omogućava koliziju vrata.

```
public Sprite unlockedSprite;  
public int lvlToLoad;  
private BoxCollider2D boxCol;
```

Unity Message | 0 references

```
void Start()  
{  
    boxCol = GetComponent<BoxCollider2D>();  
    GameManager.RegisterDoor(this);  
}
```

Unity Message | 0 references

```
private void OnTriggerEnter2D(Collider2D collision)  
{  
    if (collision.CompareTag("Player"))  
    {  
        boxCol.enabled = false;  
        collision.GetComponent<GatherInput>().DisableControls();  
  
        PlayerStats playerStats = collision.GetComponentInChildren<PlayerStats>();  
        PlayerPrefs.SetFloat("HealthKey", playerStats.health);  
  
        GameManager.ManagerLoadLevel(lvlToLoad);  
    }  
}
```

1 reference

```
public void UnlockDoor()  
{  
    GetComponent<SpriteRenderer>().sprite = unlockedSprite;  
    boxCol.enabled = true;  
}
```

Fade efekt promjene scena



Fade efekt daje ljepši i glatki prijelaz između scena

01

Duplicirati objekt Canvas i izbrisati mu djecu te dodati novi objekt „Fader”. FaderCanvas > UI > Image

02

Promijeni boju slike u crno. Fader > Inspector > Color
Rastegni sliku po ekranu: Fader > Inspector > Stretch

03

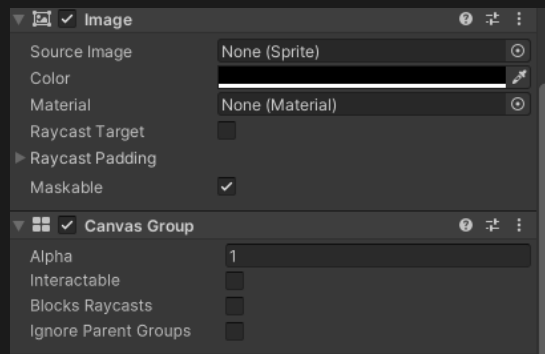
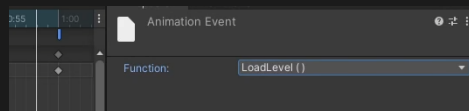
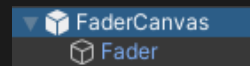
Fader > Inspector > Add Component > Canvas Group

04

Kreirati animaciju „FadeIN”. Animation > New animation > „FadeIN”
Postavi kursor na 1 sekundu i smanji u Canvas Group > Alpha na 0 te klikni na gumb „Record” u Animation prozoru

05

Kreirati animaciju „FadeOUT”. Animation > New animation > „FadeOUT”
Klikni na gumb „Record”, postavi kursor na nultu sekundi i postavi Canvas Group > Alpha na 0 te zatim postavi kursor na 1 sekundu i povećaj u Canvas Group > Alpha na 1 i postavi Animation Event na prvu sekundu i dodaj funkciju LoadLevel().



Fade efekt promjene scena

01

U Animatoru potrebno je povezati FadeIN sa FadeOUT

02

Kreirati parametar Fade tipa trigger kojeg dodajete na vezu između FadeIN i FadeOUT

03

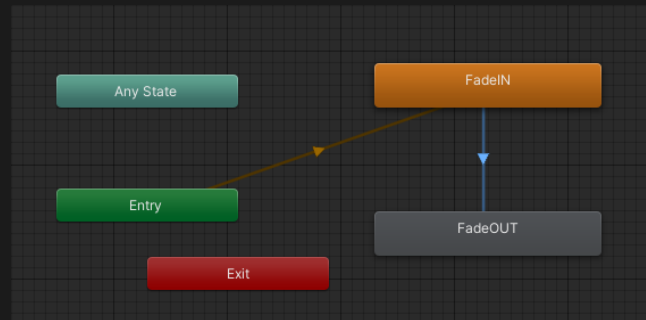
Kreirati skriptu „Fader”. Fader > Inspector > Add Component > „Fader” > New Script

04

U Metodi SetLevel postavlja se broj razine koji se učitava na temelju 'lvl' parametra i pokreće se trigger „Fade”. Metoda LoadLevel() učitava razinu igre čiji je broj pohranjen u varijabli 'lvlToLoad'. Metoda Restart() služi za ponovno pokretanje trenutne razine. Metoda RestartLevel() koristi Invoke() kako bi se odgodilo pozivanje metode Restart() za 1.5 sekundi, što daje vremena za zatamnjenje ekrana prije ponovnog pokretanja razine. Metoda ExitGame() zatvara aplikac

05

Dodati Fader objekt na objekt Door. Door > Inspector > Door Script > Fader > Fader



```
2 references
public void SetLevel(int lvl)
{
    lvlToLoad = lvl;
    anim.SetTrigger("Fade");
}

0 references
public void LoadLevel()
{
    SceneManager.LoadScene(lvlToLoad);
}

0 references
private void Restart()
{
    SetLevel(SceneManager.GetActiveScene().buildIndex);
}

1 reference
public void RestartLevel()
{
    Invoke("Restart", 1.5f);
}

0 references
public void ExitGame()
{
    Application.Quit();
}
```

Kreiranje Game Managera



Game Manager je skripta koja upravlja s kontrolama Igrača, Fadera i Door objekta.

01

Kreiraj objekt GameManager i pripadajuću skriptu „GameManager” te ju dodijelimo objektu

Metoda Awake() poziva se kada se objekt prvi put aktivira, postavlja trenutni objekt kao instancu i čuva ga tijekom cijele igre. Metoda RegisterDoor() je statička metoda koja registrira vrata i postavlja referencu na vrata 'door'

Metoda RegisterFader je statička metoda koja registrira fader i postavlja referencu na fader 'fD'

02

Metoda ManagerRestartLevel() omogućuje ponovno pokretanje trenutne razine i briše sve dijamante iz liste i poziva fader da ponovno pokrene igru

Metoda RegisterGem() registrira dijamant na vezanoj listi dijamantata, dodaje predani dijamant 'gem' na listu 'gems'

Metoda RemoveGemFromList() uklanja dijamante s liste dijamantata kada ga igrač pokupi. Ako lista postane prazna to znači da je igrač pokupio sve dijamante na razini i otključavaju se vrata pozivom metode UnlockDoor()-

Unity Message | 0 references

```
private void Awake()
{
    if (GM == null)
    {
        GM = this;
        DontDestroyOnLoad(gameObject);
    }
    else
        Destroy(gameObject);

    gems = new List<Gem>();
}
```

1 reference

```
public static void RegisterDoor(Door door)
{
    if (GM == null)
    { return; }

    GM.theDoor = door;
}
```

1 reference

```
public static void RegisterFader(Fader fD)
{
    if (GM == null)
    { return; }

    GM.fader = fD;
}
```

1 reference

```
public static void ManagerLoadLevel(int index)
{
    if (GM == null)
    { return; }
    GM.fader.SetLevel(index);
}
```

1 reference

```
public static void ManagerRestartLevel()
{
    if (GM == null)
    { return; }

    GM.gems.Clear();
    GM.fader.RestartLevel();
}
```

1 reference

```
public static void RegisterGem(Gem gem)
{
    if (GM == null)
    { return; }

    if (!GM.gems.Contains(gem))
    {
        GM.gems.Add(gem);
    }
}
```

1 reference

```
public static void RemoveGemFromList(Gem gem)
{
    if (GM == null)
    { return; }

    GM.gems.Remove(gem);
    if (GM.gems.Count == 0)
        GM.theDoor.UnlockDoor();
}
```


Kretajuće platforme



Kretajuće platforme mogu se kretati horizontalno ili vertikalno. Pomoću njih, igrač može lakše proći preko bezizlaznih rupa ili popesti se na određeni dio mape.

01

Kreirati platformu tako da dovučemo sprite 'BookShelf' na scenu i preimenujemo objekt na „Platform”. Dodati roditeljski objekt „PlatformParent”.

02

Kreirati dva prazna objekta kao djecu na objekt „PlatformParent” i promijeniti im ikonu u crveni kružić. Kreirani objekti „Point0” i „Point1” označavaju točke do kojih će se kretati platforma

03

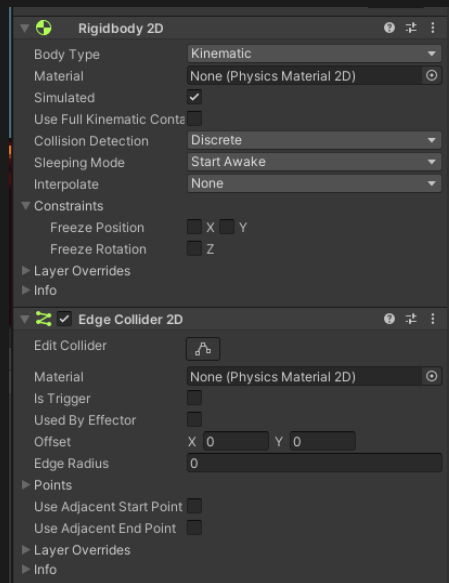
Dodati fizikalna svojstva objektu „Platform”. Dodati Rigidbody2D i EdgeCollider 2D

04

Promijeniti Layer na 'Ground'. Platform > Inspector > Layer > Ground

05

Kreirati skriptu: Platform > Inspector > Add Component > „MovingPlatform” > New Script



Krećuće platforme

Skripta nazvana „MovingPlatform” omogućuje platformi da se kreće između određenih točaka na mapi. U Platform > Inspector potrebno je podesiti brzinu kretanja platforme ('speed') i početnu te završnu točku, odnosno dodijeliti varijablama objekte 'Point0' i 'Point1'.

U startnoj metodi, postavlja se pozicija platforme na početnu točku koja je definirana prema indeksu 'startingPoint'.

U metodi Update() provjerava se da li je platforma blizu trenutne točke do koje treba doći. Ako je platforma stigla do trenutne točke, povećava se indeks 'i' kako bi se platforma kretala prema sljedećoj točki. Ako je platforma stigla do posljednje točke u polju 'points', indeks se resetira na 0 kako bi se kretala prema prvoj točki.

Metoda OnCollisionEnter2D provjerava je li igrač ušao u koliziju s platformom. Ako je, postavlja roditeljski objekt igrača na platformu, što znači da će se igrač kretati zajedno s platformom dok je na njoj

Metoda OnCollisionExit2D se poziva kada igrač izađe s kolizije s platformom. Ako je igrač izašao, uklanja se roditeljski objekt igrača s platforme, što znači da se igrač više neće kretati zajedno s platformom.

```
@ Unity Message | 0 references
void Start()
{
    transform.position = points[startingPoint].position;
}

@ Unity Message | 0 references
void Update()
{
    if(Vector2.Distance(transform.position, points[i].position) < 0.02f)
    {
        i++;
        if(i == points.Length)
        {
            i = 0;
        }
    }

    transform.position = Vector2.MoveTowards(transform.position, points[i].position, speed*Time.deltaTime);
}

@ Unity Message | 0 references
private void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.collider.CompareTag("Player"))
    {
        //if(transform.position.y < collision.transform.position.y-0.8f)
        collision.transform.SetParent(transform);
    }
}

@ Unity Message | 0 references
private void OnCollisionExit2D(Collision2D collision)
{
    if (collision.collider.CompareTag("Player"))
    {
        collision.transform.SetParent(null);
    }
}
```

Napadanje igrača



Napad igrača se kontrolira klikom lijeve tipke na mišu te time naš igrač zamahne mačem i može raditi štetu neprijateljima.

01

Kreiranje animacije napada igrača. Animation > New animation > „Attack”. Dovućemo sprite-ove za napad igrača u Animation prozor.

02

U Animatoru, spojimo „Attack” sa stanjem „Any State” i sa „Idle”. Dodajmo parametar Attack tipa bool i postavimo ga na ‘true na vezu AnyState > Attack

03

Dodajemo skriptu „PlayerAttackControls” na igrača. Player > Inspector > Add Component > „PlayerAttackControls” > New Script

04

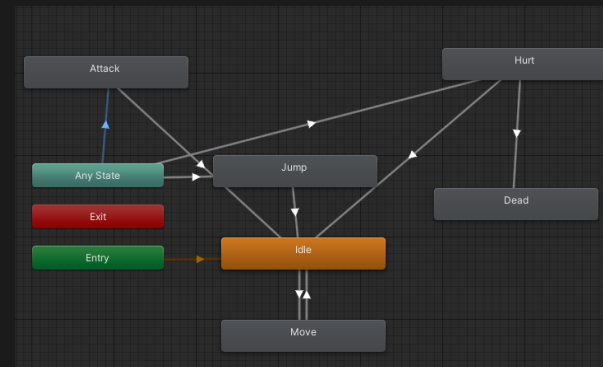
Na zadnjem frame-u u Animation prozoru Player-a dodajemo Animation Event i pridjeljujemo mu funkciju ResetAttack()

05

Postavimo sprite napadanja kako bi lakše napravili collider granice za napad. Player > Inspector > Sprite Renderer > Sprite > Attack

06

Dodajmo prazni objekt kao dijete igraču i dodajmo mu Polygon Collider 2D. Player > Create Empty > „PlayerAttack” > Inspector > Add Component > Polygon Collider 2D > IsTrigger.



Napadanje igrača

Skripta „PlayerAttackControls” upravlja akcijom napada igrača u igri.

U metodi Update() poziva se metoda Attack() koja provjerava je li igrač pokrenuo napad.

U metodi Attack() provjerava se je li igrač pokušao napasti što se provjerava pomoću varijable ‘tryAttack’. Ako je igrač pokušao napasti, provjerava se je li napad već započeo ili je igrač izgubi kontrolu nad likom. Ako ovi uvjeti nisu zadovoljeni, onda se izvršava animacija napada.

Metoda ActivateAttack() omogućuje aktiviranje kolizije tijekom napada.

Metoda ResetAttack() resetira postavke napada nakon što se animacija završi. Postavlja animaciju napada na ‘false’, označava da napad nije započeo i onemogućuje koliziju tijekom napada

```
Unity Message | 0 references
void Start()
{
    pMC = GetComponent<PlayerMoveControls>();
    anim = GetComponent<Animator>();
    gI = GetComponent<GatherInput>();
}

Unity Message | 0 references
void Update()
{
    Attack();
}

1 reference
private void Attack()
{
    if (gI.tryAttack)
    {
        if (attackStarted || pMC.hasControl == false || pMC.knockBack)
            return;

        anim.SetBool("Attack", true);
        attackStarted = true;
    }
}

0 references
public void ActivateAttack()
{
    polyCol.enabled = true;
}

1 reference
public void ResetAttack()
{
    anim.SetBool("Attack", false);
    attackStarted=false;
    polyCol.enabled = false;
}
```



Animacija umiranja neprijatelja

01

Dodati sprite za animaciju ozljeđivanja (hurt) u Animation prozor.
PatrollingEnemy > Animation > New Animation > Hurt

02

Dodati sprita za animaciju umiranja (death) u Animation prozor.
PatrollingEnemy > Animation > New Animation > DeathEnemy

03

U prozoru Animatora dodati dva trigger parametra 'Hurt' i 'Death'. 'Hurt' parametar se dodaj na vezu između RedDogWalk animacije i Hurt animacije, a 'Death' parametar se dodaje vezi između RedDogWalk animacije i DeathEnemy animacije.

04

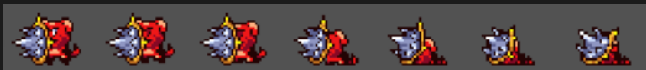
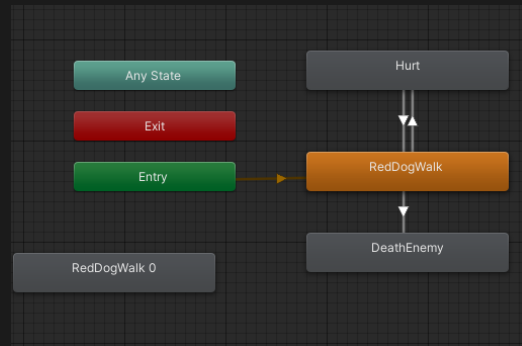
U skripti „PatrollingEnemy” potrebno je napisati dvije metode koje triggeraju parametre kod animacije neprijatelja.

05

Metoda HurtSequence() se poziva kada neprijatelj primi štetu

06

Metoda DeathSequence() se poziva kada neprijatelj umire. Postavlja brzinu neprijatelja na 0, onemogućuje fizikalna svojstva Collidera te postavlja gravitacijsko skaliranje na 0 kako bi se spriječilo propadanje kroz zemlju.



```
2 references
public override void HurtSequence()
{
    anim.SetTrigger("Hurt");
}

2 references
public override void DeathSequence()
{
    anim.SetTrigger("Death");
    speed = 0;
    GetComponent<CapsuleCollider2D>().enabled = false;
    GetComponentInChildren<PolygonCollider2D>().enabled = false;
    rb.gravityScale = 0;
}
```

Rasipanje dijamanta



Efekt rasipanja dijamanta daje bolji i ljepši prikaz prikupljanja dijamanta. Kada igrač pokupi dijamant, dijamant se rasipa u puno malih dijamanta.

01

Hierarchy > Effect > Particle System > „GemParticle”
GemParticle > Inspector > Renederer > Material > Sprites-Default

02

Podesiti Start Lifetime tako da neke čestice se unište nakon 1 sekunde, a neke nakon 2 sekunde. GemParticle > Inspector > Start Lifetime

03

Podesiti brzinu izlaženja čestica iz jezgre. GemParticle > Inspector > Start Size > 1

04

Postaviti gravitacijsko svojstvo tako da čestice padaju prema dolje. GemParticle > Inspector > Gravity Modifier > 1

05

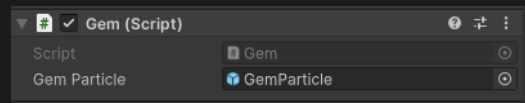
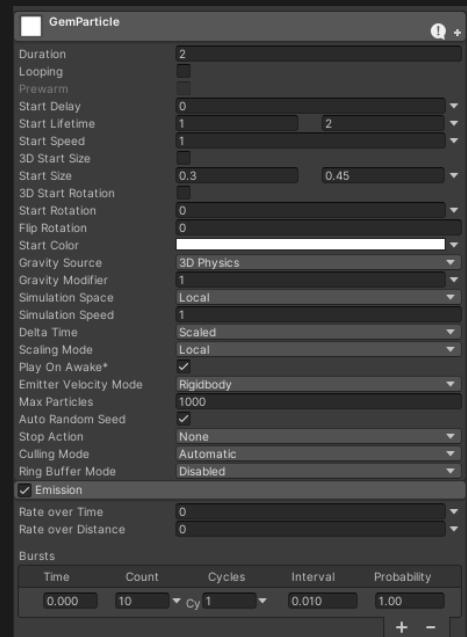
Postaviti opciju praska (burst) čestica od jednom. GemParticle > Inspector > Emission > Bursts > Count > 10

06

Postaviti dijamante za čestice. GemParticle > Inspector > Texture Sheet Animation > Mode > Sprites

07

Dodati Prefab GemParticles u skriptu „Gem” tako da kada ga igrač skupi, izvršava se animacija rasipanja dijamanta



```
Instantiate(gemParticle, transform.position, transform.rotation);
```

Kreiranje Main Menu-a



Za kreiranje početnog izbornika (Main menu) potrebni nam je Unity-ev paket TextMeshPro koji daje precizniji tekst i veću mogućnost upravljanja njim. Na početnom izborniku nalazit će se tri opcije: „Play”, „About” i „Quit”. Klikom na prvu opciju otvara se kreirana igra i učitava se prvi level. Na drugoj opciji piše zahvala i akreditacija za kreiranu igru, a klikom na treću opciju „Quit” izlazimo van iz aplikacije.

01

Canvas > UI > Text-TextMeshPro > „GameTitle”

02

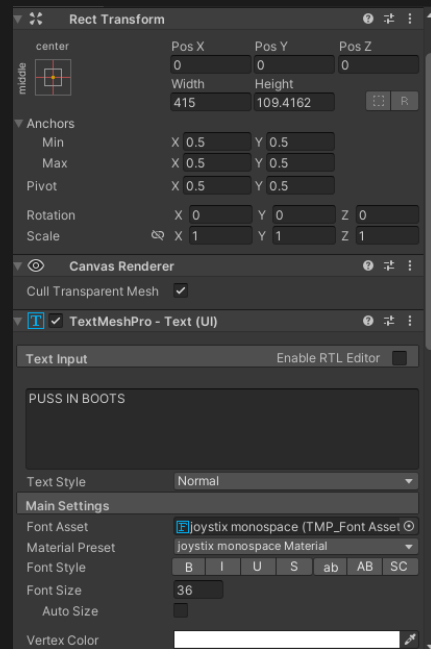
Postaviti poravnanje teksta na obostrano i importirati font naslova.
GameTitle > Inspector > Font Asset

03

Dodati pozadinu na tekst i postaviti tekst kao dijete pozadinskoj slici.
Canvas > UI > Image > Inspector > Anchor Presets > Middle left

04

Za sliku pozadine, dovući sprite health bara i promijeniti mu veličinu i boju



PUSS IN BOOTS

Kreiranje Main Menu-a

01 Kreiranje gumba. Hierarchy > UI > Button-TextMeshPro > „PlayButton”

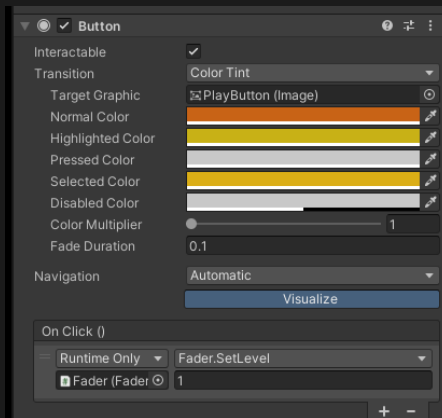
02 Napraviti funkcionalnost klika na gumb tako da dovućemo Fader objekt i odaberemo funkciju SetLevel koja postavlja broj levela iz Fader skripte. PlayButton > Inspector > OnClick() > Fader > SetLevel(int) > 1

03 Dovući healthbar i postaviti ga kao sliku pozadine te promijeniti boju i font. Postavlja se Normal Color što je boja samog gumba, „Highlighted Color što je boja kada se kursorom prijeđe preko gumba i Selected Color je boja kada se klikne na gumb

04 Postaviti prazni objekt „ButtonParent” kao dijete na ParentTitle koji će predstavljati raspored svih gumbova na sceni i biti će roditeljski objekt gumbima

05 Duplicirati gumb „PlayButton” i preimenovati u „Credits” i „Quit”

06 Postaviti automatski prvi odabir gumba. EventSystem > Inspector > FirstSelected > PlayButton



PLAY

CREDITS

QUIT



Kreiranje Main Menu-a

01

Dodati mogućnost izlaza iz igre tako da u Fader skriptu dodamo funkciju za izlaz te ju pridružimo „Quit” gumbu. PlayButton > Inspector > OnClick() > Fader > ExitGame()

```
using UnityEngine;
public void ExitGame()
{
    Application.Quit();
}
```

02

Kreirati sliku s tekstom za gumb „Credits”. Canvas > UI > Image > „CreditsPanel” > Inspector > Anchor Presets > Stretch

03

Dodati tekst na sliku. CreditsPanel > UI > Text-TextMeshPro

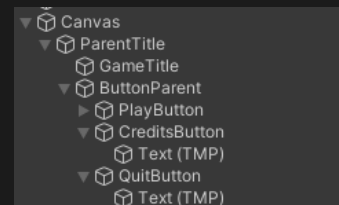
04

Dodati gumb (Back) za povratak na glavni izbornik



05

Omogućiti otvaranje CreditsPanel-a kada se klikne na gumb „Credits” tako da dovučemo objekt CreditsPanel na OnClick funkciju gumba. CreditsButton > Inspector > OnClick() > CreditsPanel > GameObject > SetActive()



Glavni izbornik

PUSS IN BOOTS

PLAY

CREDITS

QUIT



Izvoz igrice

01

File > Build Settings > Target Platform > Windows

02

Urediti postavke izvoza igre kao što su „splash image”, ikona igre i naslov završne aplikacije. File > Build Settings > Player Settings

03

Za ikonu igre uzimamo ikonu glavnog lika igrice „Puss in Boots”. Naslov igre je „Puss in Boots”.

04

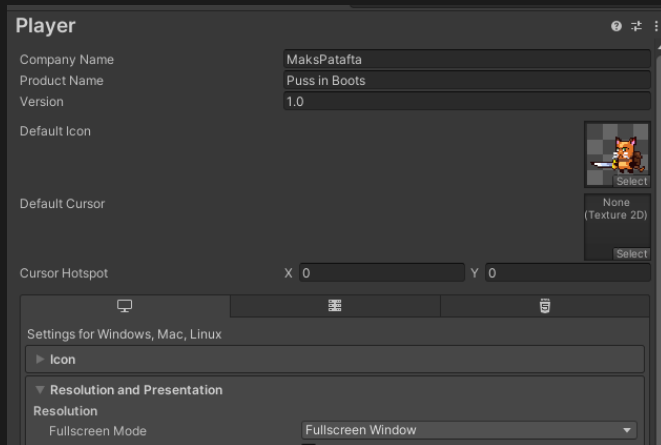
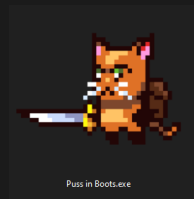
Postavljanje rezolucije igre. File > Build Settings > Player Settings > Resolution and Presentation > Fullscreen Mode > Fullscreen Window

05

Za postavljanje „Splash image” zaslona, koristimo default Unity Logo

06

Nakon što su sve postavke zadovoljene može se izvesti igra na računalu. File > Build Settings > Player Settings > Build





Hvala na
pažnji!

Maks Patafta
Razvoj računalnih igrara

